

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчет по лабораторной работе №2
по дисциплине
«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00 _____ /Черкасов А. А./
Руководитель преподаватель _____ /Шмакова Н. А./

Киров 2026

Цель работы

Цель работы: изучить и освоить принципы объектно-ориентированного программирования, включая инкапсуляцию, наследование и полиморфизм, путем создания иерархии классов в выбранной предметной области (управление гоночными командами), взаимодействия с базой данных и графическим веб-интерфейсом.

Задание

На основании согласованной в первой лабораторной работе предметной области разработать приложение с графическим интерфейсом пользователя, используя классы и особенности C#. Сведения об объектах предметной области необходимо хранить в БД.

Решение

Предметная область описывает управление гоночными командами трех различных серий (Formula 1, Formula 2, Formula E). Система позволяет хранить информацию о командах и их гонщиках, учитывать их специфические характеристики, проводить симуляцию гонок с начислением очков гонщикам и командам по правилам конкретной серии, выполнять сравнение команд по очкам через перегруженные операторы и выводить информацию о них в удобном виде.

Программа содержит классы «FormulaTeam», представляющий базовый абстрактный класс команды, «F1Team», наследованный от «FormulaTeam» и представляющий класс команды Formula 1, «F2Team», наследованный от «FormulaTeam» и представляющий класс команды Formula 2, «FormulaETeam», наследованный от «FormulaTeam» и представляющий класс команды Formula E, а также вспомогательный класс

«FormulaDriver» для хранения информации о гонщиках.

Описание структур классов

Далее в таблицах 1-5 приведено подробное описание полей и методов разработанных классов предметной области.

Таблица 1 – Описание членов базового класса FormulaTeam (Абстрактный)

Название	Тип	Доступ	Описание
Свойства			
Id	int	public	Уникальный идентификатор
TeamName	string	public	Название команды
PrincipalName	string	public	Имя руководителя команды
Headquarters	string	public	Штаб-квартира команды
TeamColor	string?	public	Цвет команды
ChampionshipPoints	int	public	Очки чемпионата
RaceWins, Podiums	int	public	Победы и подиумы
Drivers	List<FormulaDriver>	public	Список гонщиков
Методы и Конструкторы			
FormulaTeam(...)	-	protected	Конструкторы
GetSeriesName()	string	public	Абстрактный метод
CalculatePoints(pos: int)	int	public	Абстрактный метод
AddPoints(pts: int)	void	public	Добавление очков
AddWin(), AddPodium()	void	public	Изменение статистики
GetWinRate(...)	double	public	Расчет процента побед
GetSummary()	string	public	Сводка по команде (virtual)
operator >, operator <	bool	public	Сравнение команд по очкам

Таблица 2 – Описание членов класса F1Team

Название	Тип	Доступ	Описание
Свойства			
PowerUnit	string	public	Тип силовой установки
BudgetCapMln	double	public	Лимит бюджета в миллионах
ConstructorPos	int	public	Позиция в кубке конструкторов
Методы и Конструкторы			
F1Team(...)	-	public	Конструкторы
CalculateBudgetAllocation(...)	double	public	Расчет выделения бюджета
GetPowerUnitStatus()	string	public	Статус силовой установки
Переопределенные методы	разный	public	GetSeriesName, CalculatePoints, GetSummary

Таблица 3 – Описание членов класса F2Team

Название	Тип	Доступ	Описание
Свойства			
ChassisModel	string	public	Модель шасси
F1Graduates	int	public	Число выпускников в F1
IsFeederSeries	bool	public	Статус молодежной серии
Методы и Конструкторы			
F2Team(...)	-	public	Конструкторы
GetFeederStatus()	string	public	Статус молодежной команды
Переопределенные методы	разный	public	GetSeriesName, CalculatePoints, GetSummary

Таблица 4 – Описание членов класса FormulaETeam

Название	Тип	Доступ	Описание
Свойства			
SustainabilityScore	int	public	Рейтинг экологичности
BatteryCapacityKwh	double	public	Емкость батареи
EnergyPartner	string	public	Энергетический партнер
Методы и Конструкторы			
FormulaETeam(...)	-	public	Конструкторы
GetBatteryEfficiencyRating()	string	public	Рейтинг эффективности батареи
Переопределенные методы	разный	public	GetSeriesName, CalculatePoints, GetSummary

Таблица 5 – Описание членов класса FormulaDriver

Название	Тип	Доступ	Описание
Свойства			
Id	int	public	Уникальный идентификатор
TeamId	int	public	Идентификатор команды (FK)
FirstName	string	public	Имя гонщика
LastName	string	public	Фамилия гонщика
RacingNumber	int	public	Гоночный номер
Points	int	public	Набранные очки
FullName	string	public	Полное имя (вычисляемое)
DisplayString	string	public	Строка для UI (вычисляемая)
Методы и Конструкторы			
FormulaDriver(...)	-	public	Конструкторы
AddPoints(...)	void	public	Увеличение количества очков

Диаграмма классов сущностей представлена на рисунке 1.



Рисунок 1 – Диаграмма классов сущностей

Разработаны серверное приложение на бэкенде в виде REST API на C# и клиентское приложение в виде веб-интерфейса (React). Серверное приложение содержит подключение к базе данных PostgreSQL, конфигурацию моделей с использованием ТРН и маршруты API.

Подключение к базе данных происходит с помощью следующей строки подключения в методе OnConfiguring класса RacingDbContext:

```
optionsBuilder.UseNpgsql(
    "Host=localhost;Database=postgres;" +
    "Username=postgres;Password=postgres");
```

И сам класс контекста:

```
public class RacingDbContext : DbContext
{
    public DbSet<FormulaTeam> Teams { get; set; } = null!;
    public DbSet<FormulaDriver> Drivers { get; set; } = null!;

    protected override void OnConfiguring(
        DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseNpgsql(
            "Host=localhost;Database=postgres;" +
            "Username=postgres;Password=postgres");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Настройка TPH и связей
        base.OnModelCreating(modelBuilder);
    }
}
```

Конфигурация сущности FormulaTeam с использованием стратегии Table-per-Hierarchy (TPH) содержит настройку дискриминатора по полю series_id для деления на F1, F2 и Formula E:

```
modelBuilder.Entity<FormulaTeam>()
    .ToTable("racing_teams")
    .HasDiscriminator<long>("series_id")
    .HasValue<F1Team>(1)
    .HasValue<F2Team>(2)
    .HasValue<FormulaETeam>(3);
```

Следующим шагом является маршрутизация API-запросов от клиентов. Обработчик ProcessRequest в классе Program выполняет роль маршрутизатора API.

Содержит следующие методы:

- GET /api/teams – возвращает список всех команд с их гонщиками, отсортированных по очкам;
- POST /api/teams – создает новую команду;
- PUT /api/teams/{id} – обновляет существующую команду;
- DELETE /api/teams/{id} – удаляет команду и каскадно удаляет привязанных к ней гонщиков;
- POST /api/teams/{id}/simulate-race – симулирует гонку для команды, полиморфно вычисляя очки по занятому месту и сохраняя изменения в базе данных:

```
int points = team.CalculatePoints(position);  
driver.AddPoints(points);  
team.AddPoints(points);
```

- GET /api/drivers – возвращает список всех гонщиков;
- POST /api/drivers – создает запись о гонщике;
- PUT /api/drivers/{id} – обновляет запись о гонщике;
- DELETE /api/drivers/{id} – удаляет запись о гонщике по идентификатору;
- GET /api/teams/compare – метод сравнения двух команд по очкам с помощью перегруженных операторов:

```
public static bool operator >(FormulaTeam a, FormulaTeam b)  
=> a.ChampionshipPoints > b.ChampionshipPoints;
```

Примеры реализации CRUD-операций

Ниже приведены примеры C#-кода для каждой из четырех операций CRUD (Create, Read, Update, Delete) на примере сущности гоночной команды (**FormulaTeam**), реализованных на уровне API-обработчика и менеджера базы данных.

1. Create (Создание)

Пример маршрутизации POST-запроса для добавления новой команды в `Program.cs`:

```
else if (method == "POST")
{
    string body = ReadBody(request);
    var team = JsonSerializer.Deserialize<FormulaTeam>(body, jsonOptions);
    if (team == null) throw new ArgumentException("Invalid team payload");
    db.SaveTeam(team);
    SendJson(response, team, 201);
    return;
}
```

Метод `SaveTeam` в `DatabaseManager.cs` (добавление новой записи при `Id == 0`):

```
public void SaveTeam(FormulaTeam team)
{
    using var context = new RacingDbContext();
    if (team.Id == 0)
    {
        context.Teams.Add(team);
    }
    else
    {
        context.Teams.Update(team);
    }
    context.SaveChanges();
}
```

2. Read (Чтение)

Пример маршрутизации GET-запроса для получения всех команд в `Program.cs`:

```

if (method == "GET")
{
    var teams = db.GetAllTeams();
    SendJson(response, teams);
    return;
}

```

Метод `GetAllTeams` в `DatabaseManager.cs` с жадной загрузкой (Eager Loading) гонщиков:

```

public List<FormulaTeam> GetAllTeams()
{
    using var context = new RacingDbContext();
    return context.Teams
        .Include(t => t.Drivers)
        .OrderByDescending(t => t.ChampionshipPoints)
        .ToList();
}

```

3. Update (Обновление)

Пример маршрутизации PUT-запроса для обновления существующей команды в `Program.cs`:

```
if (method == "PUT")
{
    string body = ReadBody(request);
    var team = JsonSerializer.Deserialize<FormulaTeam>(body, jsonOptions);
    if (team == null) throw new ArgumentException("Invalid team payload");
    team.Id = teamId;
    db.SaveTeam(team);
    SendJson(response, team);
    return;
}
```

(Для сохранения изменений используется тот же метод `SaveTeam` с веткой `context.Teams.Update(team)` при `Id != 0`).

4. Delete (Удаление)

Пример маршрутизации DELETE-запроса в `Program.cs`:

```
else if (method == "DELETE")
{
    db.DeleteTeam(teamId);
    response.StatusCode = 204;
    response.Close();
    return;
}
```

Метод `DeleteTeam` в `DatabaseManager.cs` с каскадным удалением СВЯЗАННЫХ ГОНЩИКОВ:

```
public void DeleteTeam(int id)
{
    using var context = new RacingDbContext();
```

```

var team = context.Teams.Include(t => t.Drivers).FirstOrDefault(t =>
    t.Id == id);
if (team != null)
{
    context.Drivers.RemoveRange(team.Drivers);
    context.Teams.Remove(team);
    context.SaveChanges();
}
}

```

Диаграмма классов контроллеров и конфигураций сущностей представлены на рисунке 2.

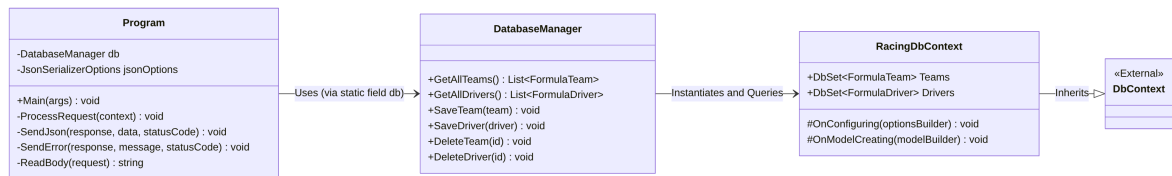


Рисунок 2 – Диаграмма классов базы данных и API

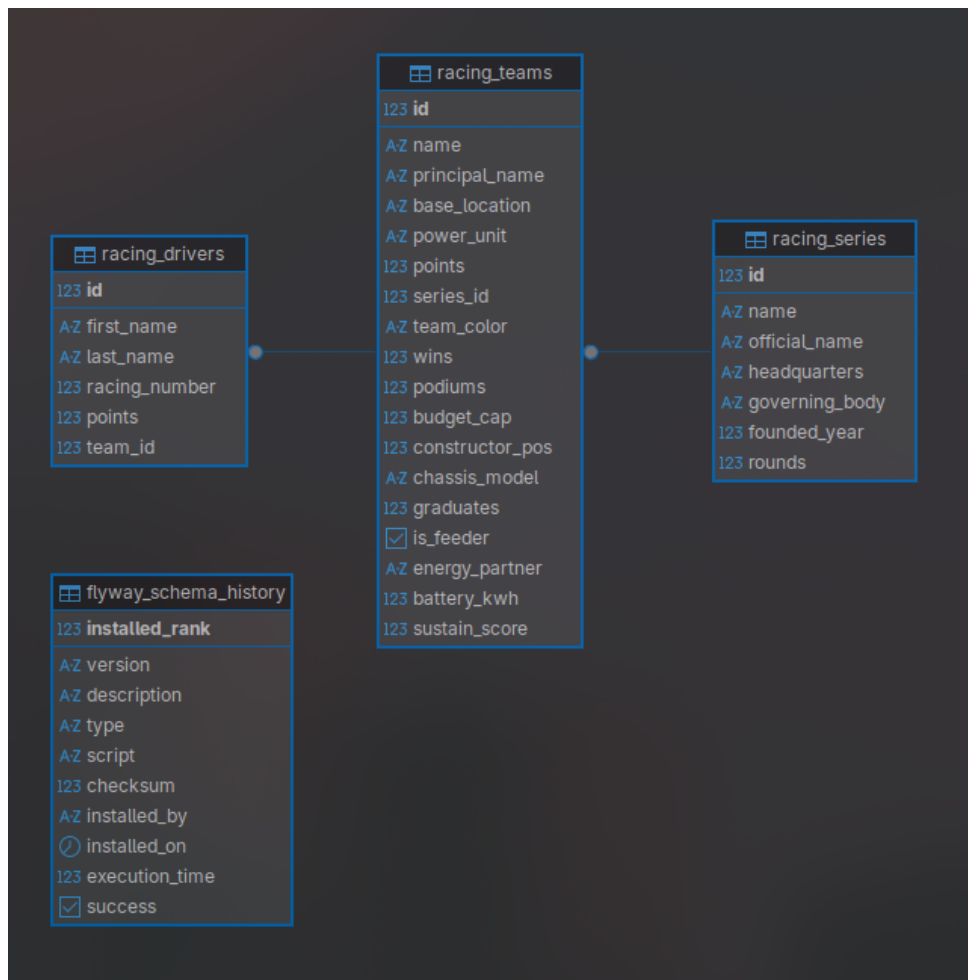


Рисунок 3 – Диаграмма ERD базы данных (схема)

Также разработан веб-интерфейс для взаимодействия с WEB API, экранные формы которого представлены на рисунках 4, 5, 6 и 7.

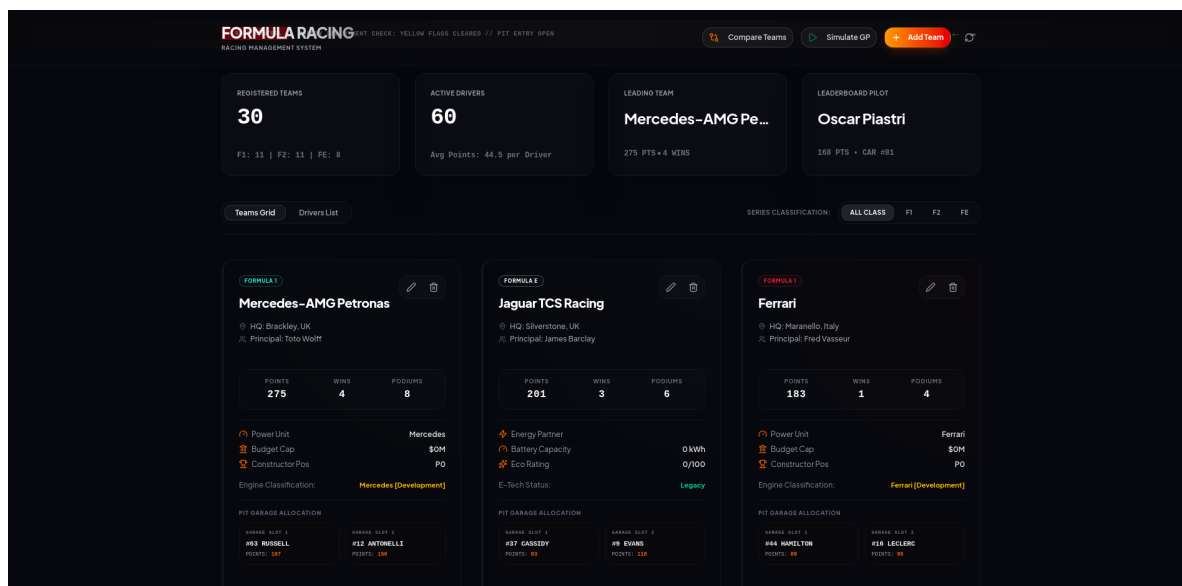


Рисунок 4 – Экранная форма вкладки команд (standings)

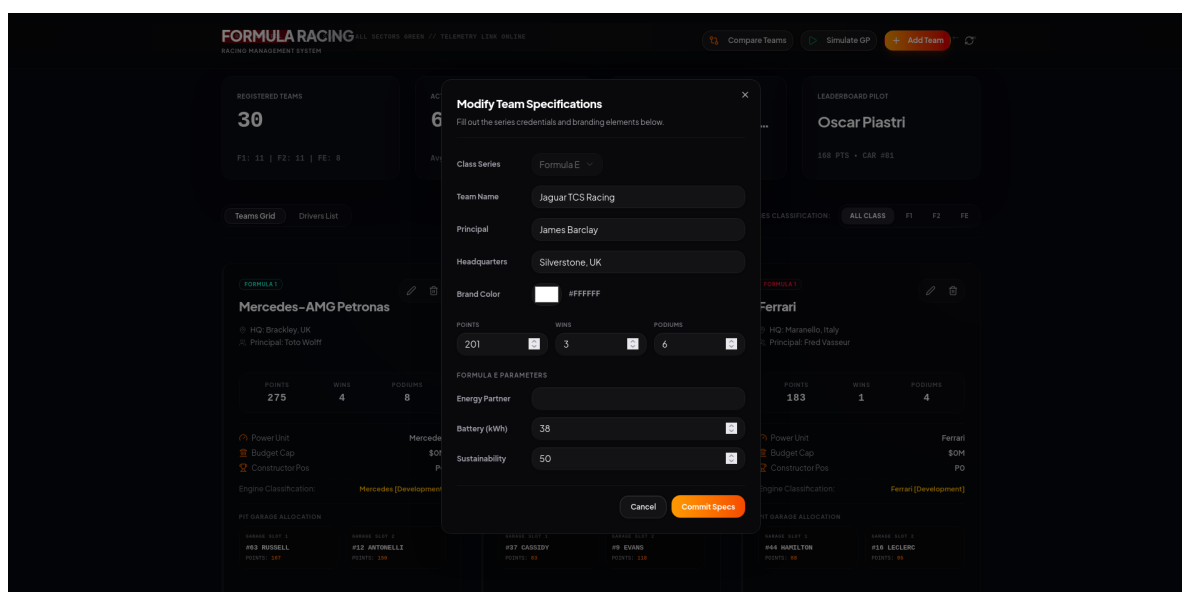


Рисунок 5 – Экранная форма вкладки гонщиков

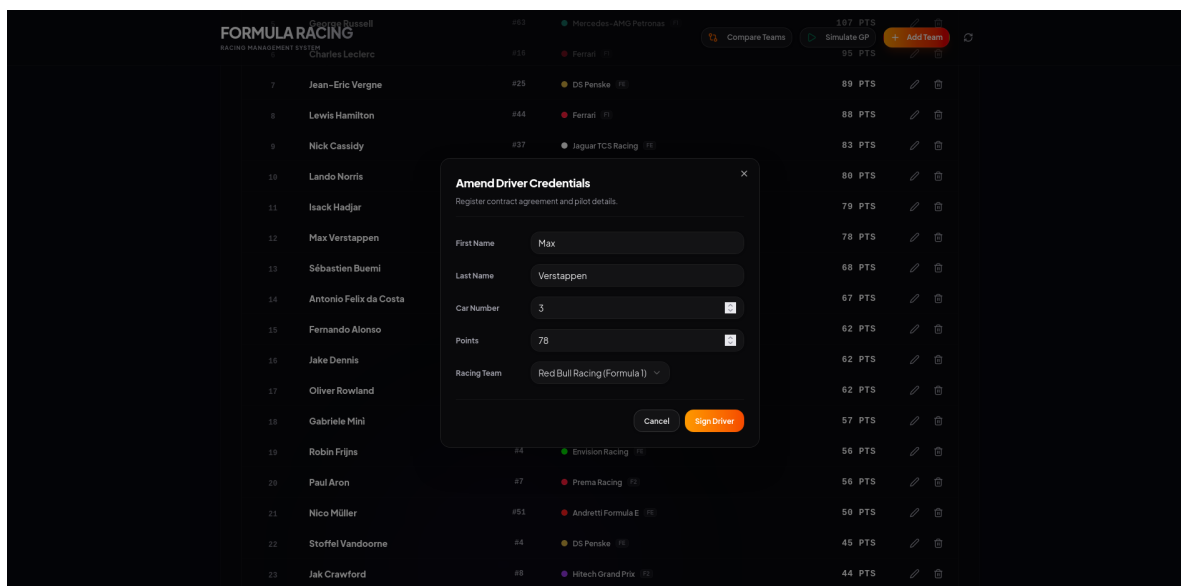


Рисунок 6 – Экранная форма создания и редактирования команд

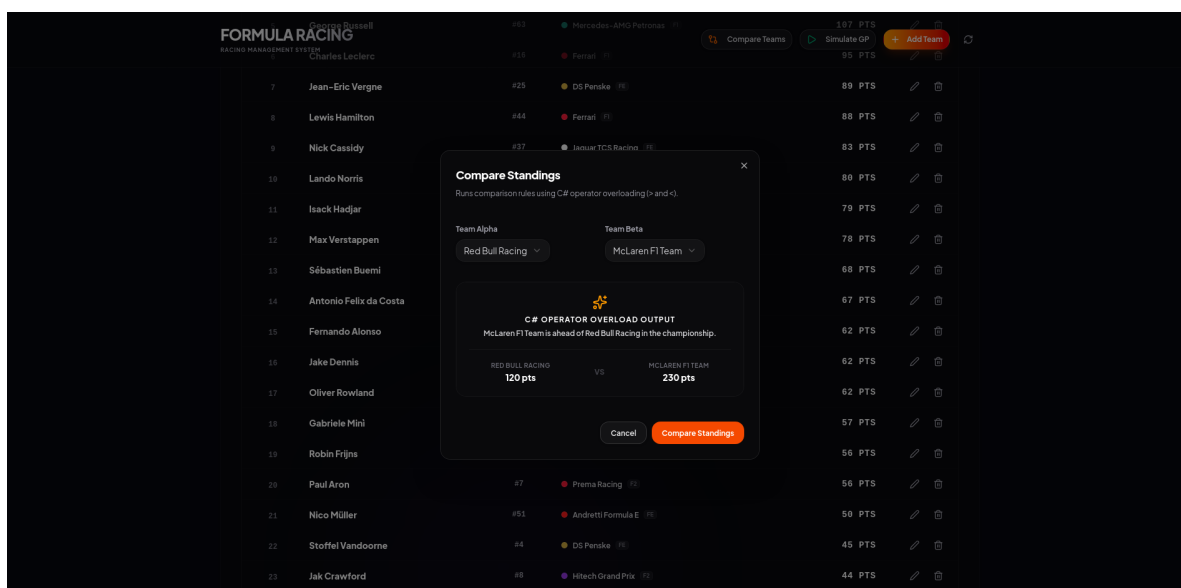


Рисунок 7 – Экранная форма создания и редактирования гонщиков

Вывод

В ходе выполнения лабораторной работы была разработана и реализована на языке C# иерархия классов, моделирующая управление гоночными командами различных серий. Изучены основные принципы объектно-ориентированного программирования: инкапсуляция, наследование, полиморфизм. Реализованы механизмы работы с реляционной

базой данных PostgreSQL через Entity Framework Core с использованием стратегии Table-per-Hierarchy (TPH). Разработано веб-приложение на базе React и бэкенда на C# REST API, демонстрирующее корректное применение данных принципов.

Приложение А1. Базовый класс FormulaTeam.cs

```
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;

namespace RacingSystem.Core
{
    [JsonPolymorphic(TypeDiscriminatorPropertyName = "type")]
    [JsonDerivedType(typeof(F1Team), "f1")]
    [JsonDerivedType(typeof(F2Team), "f2")]
    [JsonDerivedType(typeof(FormulaETeam), "fe")]
    public abstract class FormulaTeam : ITeam
    {
        // Enforce encapsulation with Properties
        public int Id { get; set; }
        public string TeamName { get; set; } = "Unknown";
        public string PrincipalName { get; set; } = "Unknown";
        public string Headquarters { get; set; } = "Unknown";
        [JsonPropertyName("team_color")]
        public string? TeamColor { get; set; }
        public int ChampionshipPoints { get; set; }
        public int RaceWins { get; set; }
        public int Podiums { get; set; }
        public List<FormulaDriver> Drivers { get; set; } = new
            ↪ List<FormulaDriver>();

        protected FormulaTeam() : this("Unknown", "Unknown", "Unknown") { }

        protected FormulaTeam(string name, string principal, string hq)
        {
            TeamName = name;
            PrincipalName = principal;
            Headquarters = hq;
        }
    }
}
```

```

protected FormulaTeam(string name, string principal, string hq, int
↪ points, int wins, int podiums)
    : this(name, principal, hq)
{
    ChampionshipPoints = points;
    RaceWins = wins;
    Podiums = podiums;
}

// Logic methods from Lab 1
public void AddPoints(int pts)
{
    if (pts > 0) ChampionshipPoints += pts;
}

public void AddWin()
{
    RaceWins++;
    Podiums++;
}

public void AddPodium() => Podiums++;

public double GetWinRate(int totalRaces)
{
    if (totalRaces <= 0) return 0.0;
    return (double)RaceWins / totalRaces * 100.0;
}

// Abstract methods (requirement from Lab 1)
// Logic methods
public abstract string GetSeriesName();
public abstract int CalculatePoints(int position);

```

```

public virtual string GetSummary()
{
    return $"[{GetSeriesName()}] {TeamName} | Pts: {ChampionshipPoints} |
    ↪ Wins: {RaceWins}";
}

// Properties for UI binding
public string SeriesName => GetSeriesName();
public string Summary => GetSummary();

[JsonPropertyName("visible_color")]
public string VisibleColor => GetVisibleTeamColor(TeamColor);

private static string GetVisibleTeamColor(string? hex)
{
    if (string.IsNullOrEmpty(hex)) return "#3b82f6";

    string cleanHex = hex.Replace("#", "").Trim();
    if (cleanHex.Equals("000000", StringComparison.OrdinalIgnoreCase) ||
        cleanHex.Equals("000", StringComparison.OrdinalIgnoreCase) ||
        cleanHex.Equals("1E293B", StringComparison.OrdinalIgnoreCase) ||
        cleanHex.Equals("18181b", StringComparison.OrdinalIgnoreCase))
    {
        return "#a1a1aa";
    }

    if (cleanHex.Length == 3)
    {
        cleanHex = $"{cleanHex[0]}{cleanHex[0]}" +
            $"{cleanHex[1]}{cleanHex[1]}" +
            $"{cleanHex[2]}{cleanHex[2]}";
    }

    if (cleanHex.Length != 6) return hex;
}

```

```

try
{
    int rVal = Convert.ToInt32(cleanHex.Substring(0, 2), 16);
    int gVal = Convert.ToInt32(cleanHex.Substring(2, 2), 16);
    int bVal = Convert.ToInt32(cleanHex.Substring(4, 2), 16);

    double r = rVal / 255.0;
    double g = gVal / 255.0;
    double b = bVal / 255.0;

    double luminance = 0.2126 * r + 0.7152 * g + 0.0722 * b;

    if (luminance < 0.35)
    {
        double max = Math.Max(r, Math.Max(g, b));
        double min = Math.Min(r, Math.Min(g, b));
        double h = 0, s = 0, l = (max + min) / 2.0;

        if (max != min)
        {
            double d = max - min;
            s = l > 0.5 ? d / (2.0 - max - min) : d / (max + min);
            if (max == r)
            {
                h = (g - b) / d + (g < b ? 6.0 : 0.0);
            }
            else if (max == g)
            {
                h = (b - r) / d + 2.0;
            }
            else if (max == b)
            {
                h = (r - g) / d + 4.0;
            }
            h /= 6.0;
        }
    }
}

```

```

    }

    l = Math.Max(l, 0.55);
    s = Math.Max(s, 0.65);

    double Hue2Rgb(double pVal, double qVal, double tVal)
    {
        if (tVal < 0) tVal += 1.0;
        if (tVal > 1.0) tVal -= 1.0;
        if (tVal < 1.0 / 6.0) return pVal + (qVal - pVal) * 6.0 * tVal;
        if (tVal < 1.0 / 2.0) return qVal;
        if (tVal < 2.0 / 3.0) return pVal + (qVal - pVal) * (2.0 / 3.0 -
        ↪ tVal) * 6.0;
        return pVal;
    }

    double q = l < 0.5 ? l * (1.0 + s) : l + s - l * s;
    double p = 2.0 * l - q;

    r = Hue2Rgb(p, q, h + 1.0 / 3.0);
    g = Hue2Rgb(p, q, h);
    b = Hue2Rgb(p, q, h - 1.0 / 3.0);

    string ToHex(double val)
    {
        int intVal = (int)Math.Round(val * 255.0);
        return intVal.ToString("X2").ToLower();
    }

    return $"#{ToHex(r)}{ToHex(g)}{ToHex(b)}";
}
}
catch
{
    return hex;
}

```



```

    }

    return hex;
}

// Operator overloading
public static bool operator >(FormulaTeam a, FormulaTeam b) =>
    ⇨ a.ChampionshipPoints > b.ChampionshipPoints;
public static bool operator <(FormulaTeam a, FormulaTeam b) =>
    ⇨ a.ChampionshipPoints < b.ChampionshipPoints;

public override string ToString() => GetSummary();
}
}

```

Приложение A2. Класс F1Team.cs

```

using System;

namespace RacingSystem.Core
{
    public class F1Team : FormulaTeam
    {
        public string PowerUnit { get; set; } = "Unknown";
        public double BudgetCapMln { get; set; } = 135.0;
        public int ConstructorPos { get; set; }

        public F1Team() : this("Unknown", "Unknown", "Unknown", "Unknown") { }

        public F1Team(string name, string principal, string hq, string
            ⇨ powerUnit)
            : base(name, principal, hq)
        {
            PowerUnit = powerUnit;
        }
    }
}

```

```

public F1Team(string name, string principal, string hq, string
    ↪ powerUnit, double budget, int ctorPos, int points, int wins, int
    ↪ podiums)
    : base(name, principal, hq, points, wins, podiums)
{
    PowerUnit = powerUnit;
    BudgetCapMln = budget;
    ConstructorPos = ctorPos;
}

public override string GetSeriesName() => "Formula 1";

public override int CalculatePoints(int position)
{
    int[] points = { 25, 18, 15, 12, 10, 8, 6, 4, 2, 1 };
    return (position >= 1 && position <= 10) ? points[position - 1] : 0;
}

public double CalculateBudgetAllocation(double percentage) =>
    ↪ BudgetCapMln * (percentage / 100.0);

public string GetPowerUnitStatus()
{
    if (RaceWins > 10) return $"{PowerUnit} [Dominant]";
    if (RaceWins > 4) return $"{PowerUnit} [Competitive]";
    return $"{PowerUnit} [Development]";
}

[System.Text.Json.Serialization.JsonPropertyName("engine_status")]
public string EngineStatus => GetPowerUnitStatus();

public override string GetSummary()
{

```

```

        return $"{base.GetSummary()} | PU: {PowerUnit} | Budget:
        ↪  ${BudgetCapMln}M";
    }
}
}

```

Приложение А3. Класс F2Team.cs

```

using System;

namespace RacingSystem.Core
{
    public class F2Team : FormulaTeam
    {
        public string ChassisModel { get; set; } = "Dallara F2 2024";
        public int F1Graduates { get; set; }
        public bool IsFeederSeries { get; set; } = true;

        public F2Team() : this("Unknown", "Unknown", "Unknown", "Dallara F2
        ↪  2024") { }

        public F2Team(string name, string principal, string hq, string chassis)
            : base(name, principal, hq)
        {
            ChassisModel = chassis;
        }

        public F2Team(string name, string principal, string hq, string chassis,
        ↪  int graduates, bool feeder, int points, int wins, int podiums)
            : base(name, principal, hq, points, wins, podiums)
        {
            ChassisModel = chassis;
            F1Graduates = graduates;
            IsFeederSeries = feeder;
        }
    }
}

```

```

public override string GetSeriesName() => "Formula 2";

public override int CalculatePoints(int position)
{
    int[] points = { 15, 12, 10, 8, 6, 5, 4, 3, 2, 1 };
    return (position >= 1 && position <= 10) ? points[position - 1] : 0;
}

public string GetFeederStatus()
{
    if (!IsFeederSeries) return "Independent";
    if (F1Graduates >= 5) return "Elite Feeder (5+ F1 graduates)";
    if (F1Graduates >= 2) return "Active Feeder";
    return "Developing Feeder";
}

[System.Text.Json.Serialization.JsonPropertyName("feeder_status_text")]
public string FeederStatusText => GetFeederStatus();

public override string GetSummary()
{
    return $"{base.GetSummary()} | Chassis: {ChassisModel} | F1 Grads:
    ↪ {F1Graduates}";
}
}
}

```

Приложение A4. Класс FormulaETeam.cs

```

using System;

namespace RacingSystem.Core
{
    public class FormulaETeam : FormulaTeam

```

```

{
    public int SustainabilityScore { get; set; } = 50;
    public double BatteryCapacityKwh { get; set; } = 38.0;
    public string EnergyPartner { get; set; } = "Unknown";

    public FormulaETeam() : this("Unknown", "Unknown", "Unknown", "Unknown")
        ↪ { }

    public FormulaETeam(string name, string principal, string hq, string
        ↪ energyPartner)
        : base(name, principal, hq)
    {
        EnergyPartner = energyPartner;
    }

    public FormulaETeam(string name, string principal, string hq, string
        ↪ energyPartner, int sustainScore, double batteryKwh, int points, int
        ↪ wins, int podiums)
        : base(name, principal, hq, points, wins, podiums)
    {
        EnergyPartner = energyPartner;
        SustainabilityScore = sustainScore;
        BatteryCapacityKwh = batteryKwh;
    }

    public override string GetSeriesName() => "Formula E";

    public override int CalculatePoints(int position)
    {
        int[] points = { 25, 18, 15, 12, 10, 8, 6, 4, 2, 1 };
        return (position >= 1 && position <= 10) ? points[position - 1] : 0;
    }

    public string GetBatteryEfficiencyRating()
    {

```

```

        if (BatteryCapacityKwh >= 40.0) return "Next-Gen (40+ kWh)";
        if (BatteryCapacityKwh >= 38.0) return "Gen3 Standard (38 kWh)";
        return "Legacy";
    }

    ↪ [System.Text.Json.Serialization.JsonPropertyName("battery_rating_text")]
    public string BatteryRatingText => GetBatteryEfficiencyRating();

    public override string GetSummary()
    {
        return $"{base.GetSummary()} | Eco: {SustainabilityScore}/100 |
        ↪ Battery: {BatteryCapacityKwh} kWh";
    }
}
}

```

Приложение А5. Класс FormulaDriver.cs

```

using System;

namespace RacingSystem.Core
{
    public class FormulaDriver : IDriver
    {
        public int Id { get; set; }
        public int TeamId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int RacingNumber { get; set; }
        public int Points { get; set; }

        public FormulaDriver() : this(string.Empty, string.Empty, 0, 0) { }
    }
}

```

```

public FormulaDriver(string firstName, string lastName, int number, int
↪ points = 0)
{
    FirstName = firstName;
    LastName = lastName;
    RacingNumber = number;
    Points = points;
}

public string FullName => $"{FirstName} {LastName}";
public string DisplayString => $"{FullName} #{RacingNumber}";

public void AddPoints(int pts) => Points += pts;
}
}

```